Notes: Measuring Reinforcement Learning Against the Exact LQR Optimum

Jon Legasa

November 9, 2026

Summary

LQR has become a standard for measuring RL algorithms, but we found that some algorithms have not been tested on it, and we are trying to see the impact of LQR across many seeds to see the behavior of these algorithms. We seek to understand whether there is any way to measure the performance of these algorithms with respect to each other, and then to measure their performance on RL environments to see if these differences in performance hold even for many seeds.

1 Motivation

We came together to try to find what makes these algorithms special, following consistent benchmarks and baselines. We set up the LQR step to motivate the baseline (Section 3). Agyei, Sarhadi and Polani (2025) introduced LQI as the baseline for comparing deep RL algorithms on tracking problems in continuous time, one of them nonlinear (Section 4). From this baseline we extend to those setups.

We then introduce the many seeds, the expected value and the average of the learning curves (Section 6), because we expect the learning curves to converge to the same value across seeds, and to be bounded. Think of the set of seeds as points on a line, any of which can be picked. As we observe all seeds and measure the performance of every algorithm, it could be possible that, as training time increases, for all seeds on this line the algorithms generate learning curves that satisfy an ordering inequality. That is the setup of these notes.

The LQR is a standard diagnostic for RL because its optimal controller is known (Recht 2019). Earlier LQR comparisons covered certainty equivalence, least-squares policy iteration, REINFORCE and random search (Recht 2019; Mania, Guy and Recht 2018). Agyei et al. compared deep RL methods with an LQI controller using step-response metrics.

2 Problem and notation

$$x_{t+1} = Ax_t + Bu_t, \qquad c(x,u) = x^\top Qx + u^\top Ru, \qquad J(\pi) = \sum_{t=0}^{\infty} c(x_t, u_t).$$

Here x_t is the state at time t and u_t is the control; A describes how the state evolves on its own and B how the control acts on it. The **stage cost** is the cost paid at time t:

$$c(x_t, u_t) = \underbrace{x_t^\top Q\, x_t}_{\text{distance from 0}} + \underbrace{u_t^\top R\, u_t}_{\text{control effort}}, \qquad Q \succeq 0, \quad R \succ 0.$$

In the scalar case, A, B, Q and R are the numbers a, b, q and r.

$J(\pi)$ is the total cost of following the policy π from x_0: run the system under π and add up the stage costs. The system is noise-free and undiscounted, with fixed x_0 (Section 8 covers noise and discounting). Policies are linear, $u = -Kx$, and $J^* = \min_\pi J(\pi)$ is attained at K^*, as derived in Section 3. The agent receives the reward $r_t = -c(x_t, u_t)$ and does not know A, B, Q, R, K^* or J^*; these are used only to evaluate it. Algorithms A, B and C (PPO, SAC and TD3) are trained with many seeds, and N counts training iterations.

3 The Riccati solution

The Riccati equation is the Bellman equation of the LQR, reduced to an equation for a single matrix. The optimal cost-to-go $V(x)$, the total cost of acting optimally from state x, satisfies

$$V(x) = \min_u \Big[\, x^\top Q x + u^\top R u + V(Ax + Bu) \,\Big].$$

Guess that the cost-to-go is quadratic, with a symmetric positive semidefinite matrix P; the guess is confirmed if it leads to a consistent equation for P. Substituting it, the bracket becomes

$$V(x) = x^\top P x, \qquad x^\top Q x + u^\top R u + (Ax + Bu)^\top P (Ax + Bu).$$

This is a convex quadratic in the control, because R is positive definite and the term with P is positive semidefinite. Its gradient with respect to the control is

$$\nabla_u = 2Ru + 2B^\top P(Ax + Bu) = 2\big(R + B^\top P B\big)u + 2B^\top P A\, x,$$

and setting it to zero gives the optimal control:

$$u^* = -\big(R + B^\top P B\big)^{-1} B^\top P A\, x = -K^* x.$$

Substituting the optimal control back, both sides of the Bellman equation are quadratic forms in the state:

$$x^\top P x = x^\top \Big[\, Q + K^{*\top} R K^* + (A - BK^*)^\top P (A - BK^*) \,\Big] x.$$

Expanding the last term and grouping the two terms that contain the gain twice:

$$Q + A^\top P A - A^\top P B K^* - K^{*\top} B^\top P A + K^{*\top} \big(R + B^\top P B\big) K^*.$$

The definition of the optimal gain gives the identity below, which makes the last term equal to the one before it, so those two cancel:

$$\big(R + B^\top P B\big) K^* = B^\top P A \quad \Longrightarrow \quad K^{*\top} \big(R + B^\top P B\big) K^* = K^{*\top} B^\top P A.$$

What remains is the matrix in brackets, and it must equal P because the two quadratic forms agree for every state and both matrices are symmetric. This is the discrete algebraic Riccati equation, shown with the optimal gain and the optimal cost:

$$P = Q + A^\top P A - A^\top P B \big(R + B^\top P B\big)^{-1} B^\top P A,$$
$$K^* = \big(R + B^\top P B\big)^{-1} B^\top P A, \qquad J^* = x_0^\top P x_0.$$

P is the cost-to-go matrix: the optimal cost is the cost-to-go at the initial state, which is why the optimal cost is the quadratic form of P at the initial state. The equation is algebraic because, over an infinite horizon, P does not change with time; it is a fixed point of the Bellman equation. It is quadratic in P.

The scalar case. With $n = m = 1$:

$$A = [a], \quad B = [b], \quad Q = [q], \quad R = [r], \quad P = [p].$$

Substituting term by term into the Riccati equation, with transposes acting trivially on numbers:

$$P = Q + \underbrace{A^\top PA}_{(1)} - \underbrace{A^\top PB}_{(2)} \underbrace{\big(R + B^\top PB\big)^{-1}}_{(3)} \underbrace{B^\top PA}_{(4)},$$

(1) $A^\top PA = a\,p\,a = a^2p$, (2) $A^\top PB = a\,p\,b = ab\,p$,

(3) $\big(R + B^\top PB\big)^{-1} = \dfrac{1}{r + b\,p\,b} = \dfrac{1}{r + b^2p}$, (4) $B^\top PA = b\,p\,a = ab\,p$.

Multiplying (2), (3) and (4) gives the scalar Riccati equation:

$$(2)(3)(4) = ab\,p \cdot \frac{1}{r + b^2p} \cdot ab\,p = \frac{a^2b^2p^2}{r + b^2p}, \qquad p = q + a^2p - \frac{a^2b^2p^2}{r + b^2p}.$$

With $a = b = q = r = 1$ and $x_0 = 1$, taking the root with $p \geq 0$:

$$p = 1 + p - \frac{p^2}{1+p} \implies 0 = 1 - \frac{p^2}{1+p} \implies p^2 = 1 + p \implies p^2 - p - 1 = 0 \implies p = \frac{1+\sqrt{5}}{2},$$

$$K^* = \frac{ab\,p}{r + b^2p} = \frac{p}{1+p} = \frac{\sqrt{5}-1}{2} \approx 0.618, \qquad J^* = p\,x_0^2 = \frac{1+\sqrt{5}}{2} \approx 1.618.$$

So the cost-to-go matrix is the 1×1 matrix $[p]$, and $V(x) = px^2$. Note $(K^*)^2 + K^* - 1 = 0$ and $K^*J^* = 1$. In higher dimensions P stays a genuine matrix, and the Riccati equation is a system of coupled quadratic equations in its entries, usually solved numerically.

4 From LQR to LQI

LQR drives the state to zero. A tracking problem asks an output to follow a reference instead, and with a constant disturbance LQR leaves a permanent error. LQI is LQR applied to a system enlarged by one state that accumulates the error.

The offset under a constant disturbance. Add a constant disturbance to the system and apply the LQR controller:

$$x_{t+1} = Ax_t + Bu_t + d, \qquad u_t = -Kx_t.$$

At steady state the state stops changing, so it must satisfy

$$x_{\text{ss}} = (A - BK)\,x_{\text{ss}} + d \quad \implies \quad x_{\text{ss}} = \big(I - A + BK\big)^{-1} d \neq 0.$$

In the scalar example the steady state is a multiple of the disturbance:

$$x_{\text{ss}} = (1 - k)\,x_{\text{ss}} + d \quad \implies \quad k\,x_{\text{ss}} = d \quad \implies \quad x_{\text{ss}} = \frac{d}{k} \approx 1.618\,d \quad (k = K^*).$$

The controller pushes back only in proportion to the current error, so it needs a nonzero error to produce the push that cancels the disturbance.

Tracking as shifted regulation. To make an output follow a constant reference, find the equilibrium that gives that output, a state and a constant control that hold it:

$$y_t = Cx_t, \qquad x_{\text{ss}} = Ax_{\text{ss}} + Bu_{\text{ss}} + d, \qquad Cx_{\text{ss}} = y_{\text{ref}}.$$

Subtracting the equilibrium equation from the dynamics removes the disturbance and leaves a pure LQR problem in shifted coordinates:

$$\tilde{x}_t = x_t - x_{\text{ss}}, \quad \tilde{u}_t = u_t - u_{\text{ss}} \quad \Longrightarrow \quad \tilde{x}_{t+1} = A\tilde{x}_t + B\tilde{u}_t,$$
$$u_t = u_{\text{ss}} - K\,(x_t - x_{\text{ss}}).$$

So tracking is regulation around a shifted equilibrium, but computing that equilibrium requires knowing the disturbance and the model exactly. Without the disturbance the shift is wrong and the offset returns.

The integral state. Instead of computing the shift, add a state that accumulates the tracking error:

$$\epsilon_t = y_{\text{ref}} - Cx_t, \qquad z_{t+1} = z_t + \epsilon_t \quad \Longrightarrow \quad z_t = \sum_{s=0}^{t-1} \epsilon_s.$$

Here time is discrete, so z is a sum of past errors. In continuous time, where t is a real number, the same state is an integral, which is where the name comes from:

$$\dot{z}(t) = \epsilon(t) \quad \Longrightarrow \quad z(t) = \int_0^t \epsilon(s)\,ds.$$

The augmented LQR problem. Treat z as an extra state component. The enlarged system is still linear and the cost is still quadratic, with one more weight:

$$\begin{pmatrix} x_{t+1} \\ z_{t+1} \end{pmatrix} = \begin{pmatrix} A & 0 \\ -C & I \end{pmatrix} \begin{pmatrix} x_t \\ z_t \end{pmatrix} + \begin{pmatrix} B \\ 0 \end{pmatrix} u_t + \begin{pmatrix} d \\ y_{\text{ref}} \end{pmatrix}, \qquad \sum_t \left(x_t^\top Q x_t + z_t^\top Q_z z_t + u_t^\top R u_t \right).$$

The Riccati equation of Section 3, applied to these larger matrices in the shifted coordinates, gives a gain with one part for each state, so the LQI controller is proportional plus integral:

$$u_t = -K_x x_t - K_z z_t.$$

Zero steady-state error. At steady state every state stops changing. For the integral state this forces the error to zero:

$$z_{\text{ss}} = z_{\text{ss}} + \left(y_{\text{ref}} - Cx_{\text{ss}} \right) \quad \Longrightarrow \quad Cx_{\text{ss}} = y_{\text{ref}}.$$

In the scalar example, with output equal to the state, the two steady-state equations give

$$x_{\text{ss}} = y_{\text{ref}}, \qquad x_{\text{ss}} = x_{\text{ss}} + u_{\text{ss}} + d \implies u_{\text{ss}} = -d = -K_x y_{\text{ref}} - K_z z_{\text{ss}} \implies z_{\text{ss}} = \frac{d - K_x y_{\text{ref}}}{K_z}.$$

The integral state settles at exactly the value that cancels the disturbance, without the disturbance ever being known. The proportional part no longer needs a nonzero error, because the accumulated error holds the push.

Continuous time. The same construction holds with the dynamics written as differential equations, and the gain then comes from the continuous algebraic Riccati equation:

$$\dot{x} = Ax + Bu + d, \qquad \dot{z} = y_{\text{ref}} - Cx, \qquad A^\top P + PA - PBR^{-1}B^\top P + Q = 0.$$

The Agyei et al. environments. Agyei, Sarhadi and Polani (2025) chose LQI because their tasks are setpoint tracking; in their words, "the integral action ensures zero steady-state error for step references, making LQI superior to LQR for tracking applications." Their deep RL agents were trained on the negative of the LQI cost. All four environments are continuous-time tracking problems, and only one is nonlinear:

Environment	Model	Task
Chemical reactor (CSTR)	Linear transfer function, non-minimum phase	Step setpoint tracking
Two-mass spring (ACC 1992 benchmark)	Linear state space	Step setpoint tracking
REMUS underwater vehicle (yaw)	Nonlinear	Step setpoint tracking
Crazyflie quadrotor (altitude)	Reduced-order linear model of a nonlinear system	Multi-step altitude tracking, simulation and hardware

These notes start from the regulation baseline of Sections 2 and 3, where the optimum is exact, and the LQI construction above is how that baseline extends to these tracking environments.

5 The learning curve of one run

Cost of a linear policy. For the policy with gain k, with all parameters equal to one and initial state one, the trajectory and the stage costs are

$$u_t = -k\,x_t, \qquad x_{t+1} = x_t + u_t = (1-k)\,x_t \implies x_t = (1-k)^t,$$
$$c(x_t, u_t) = x_t^2 + u_t^2 = (1+k^2)\,x_t^2 = (1+k^2)(1-k)^{2t}.$$

The stage costs form a geometric series. It converges exactly when its ratio is below one, which is when the closed loop is stable:

$$\sum_{t=0}^{\infty} r^t = \frac{1}{1-r} \;\; (|r|<1), \qquad r = (1-k)^2 < 1 \iff 0 < k < 2, \qquad 1-(1-k)^2 = 2k-k^2 = k(2-k).$$

So the cost of a gain is

$$J : \mathbb{R} \to [0,\infty], \qquad J(k) = \sum_{t=0}^{\infty}(1+k^2)(1-k)^{2t} = \begin{cases} \dfrac{1+k^2}{k(2-k)}, & 0<k<2, \\ +\infty, & \text{otherwise.} \end{cases}$$

Functions. The gap of a gain, and, for one run of an algorithm, its gain, learning curve and perturbation, for $X \in \{A, B, C\}$:

$$\begin{aligned} g &: \mathbb{R} \to [0, \infty], & g(k) &= \frac{J(k) - J(K^*)}{J(K^*)}, \\ k_X &: \mathbb{N} \to \mathbb{R}, & k_X(N) &= \text{gain after } N \text{ iterations}, \\ g_X &: \mathbb{N} \to [0, \infty], & g_X(N) &= g\big(k_X(N)\big), \\ e_X &: \mathbb{N} \to \mathbb{R}, & e_X(N) &= k_X(N) - K^*. \end{aligned}$$

Only the gain changes during training; the optimal cost is fixed.

A simplification in this instance. Splitting the fraction, the gap is the ratio of the costs minus one:

$$g(k) = \frac{J(k) - J^*}{J^*} = \frac{J(k)}{J^*} - 1.$$

With all parameters equal to one, the optimal gain and the optimal cost are reciprocals:

$$K^* J^* = \frac{\sqrt{5} - 1}{2} \cdot \frac{1 + \sqrt{5}}{2} = \frac{5 - 1}{4} = 1 \quad \Longrightarrow \quad \frac{1}{J^*} = K^*.$$

So dividing by the optimal cost is the same as multiplying by the optimal gain:

$$g(k) = K^* J(k) - 1.$$

This holds only for these parameter values, where the optimal cost is the golden ratio and the optimal gain is its reciprocal; in general only the first form holds.

Shape of the gap.

$$\begin{aligned} g(K^*) = 0, \qquad & g \text{ decreasing on } (0, K^*), \quad \text{increasing on } (K^*, 2), \\ & g(k) \to \infty \text{ as } k \to 0^+ \text{ or } k \to 2^-. \end{aligned}$$

These follow from the derivative of the cost. Write it as a ratio and differentiate:

$$J = \frac{f}{h}, \quad f(k) = 1 + k^2, \quad h(k) = k(2 - k), \qquad J' = \frac{f'h - fh'}{h^2},$$
$$f'h - fh' = 2k \cdot k(2 - k) - (1 + k^2)(2 - 2k) = 2\,(k^2 + k - 1).$$

Since the denominator is positive, the derivative has the sign of the quadratic factor, whose only root between zero and two is

$$k^2 + k - 1 = 0 \implies k = \frac{\sqrt{5} - 1}{2} = K^*,$$

which recovers the Riccati gain. The derivative is negative to the left of this root and positive to the right. At the walls the denominator goes to zero while the numerator stays positive, so the cost goes to infinity.

Geometrically, the gap is a valley with its bottom at the optimal gain and walls at zero and two. The perturbation is the horizontal distance from the bottom, and the gap is the height. The left wall is closer to the bottom than the right wall, so the valley is steeper on the left.

Gap and perturbation. Two facts connect the size of the gap to the size of the perturbation.

A small perturbation gives a small gap. Between zero and two the cost is a ratio of polynomials with a nonzero denominator, so the gap is continuous there. Continuity at the optimal gain, where the gap is zero, says exactly:

$$\forall \varepsilon > 0 \; \exists \delta > 0 : \quad |k - K^*| < \delta \implies g(k) < \varepsilon.$$

A small gap gives a small perturbation. This holds because the optimal gain is the unique minimizer:

$$\forall \eta > 0 \; \exists \varepsilon > 0 : \quad g(k) < \varepsilon \implies |k - K^*| < \eta.$$

To find such an ε, take the smaller of the two gaps at distance η from the optimal gain, for η smaller than the optimal gain:

$$\varepsilon = \min \left\{ g(K^* - \eta),\ g(K^* + \eta) \right\} > 0.$$

The gap decreases toward the optimal gain from the left and increases away from it on the right, so every gain at distance η or more has a gap of at least ε. A gap below ε therefore forces a distance below η.

Applying the two facts along the sequence of gains of a run: once the learning curve stays below ε, the perturbation stays below η, and once the perturbation stays below δ, the learning curve stays below ε. This gives

$$g_X(N) \to 0 \iff k_X(N) \to K^* \iff e_X(N) \to 0.$$

Both facts hold for any continuous gap with a unique minimum, not only for this example.

Expansion near the optimal gain. Differentiate the identity $f = Jh$ twice and evaluate at the optimal gain, where the first derivative of the cost vanishes:

$$f'' = J''h + 2J'h' + Jh'' \quad \implies \quad J''(K^*) = \frac{f''(K^*) - J^* h''(K^*)}{h(K^*)} = \frac{2 + 2J^*}{h(K^*)}.$$

Using the Riccati identity for the optimal gain to simplify the denominator:

$$(K^*)^2 = 1 - K^* \implies h(K^*) = 2K^* - (K^*)^2 = 3K^* - 1 = \frac{3\sqrt{5} - 5}{2},$$

$$J''(K^*) = \frac{2\,(3 + \sqrt{5})}{3\sqrt{5} - 5} = \frac{7\sqrt{5} + 15}{5} \approx 6.130.$$

Taylor's theorem at the minimum, where the first derivative vanishes, gives the gap to second order:

$$g(K^* + e) = \frac{J(K^* + e)}{J^*} - 1 = \frac{J''(K^*)}{2J^*}\, e^2 + O(e^3),$$

$$\frac{J''(K^*)}{2J^*} = \frac{7\sqrt{5} + 15}{5\,(1 + \sqrt{5})} = 1 + \frac{2}{\sqrt{5}} \approx 1.894.$$

The higher-order terms were bounded numerically on a neighbourhood of the optimal gain:

$$1.758\, e^2 \le g(K^* + e) \le 2.108\, e^2 \qquad \text{for } |e| \le 0.1.$$

The bounds give explicit choices in the two facts. For the first, the upper bound controls the gap by the perturbation; for the second, the lower bound and the monotonicity above control the perturbation by the gap:

$$|e| < \delta = \sqrt{\varepsilon / 2.108} \le 0.1 \implies g(K^* + e) \le 2.108\, e^2 < 2.108\, \delta^2 = \varepsilon,$$

$$\eta \le 0.1,\ \varepsilon = 1.758\,\eta^2: \quad |e| \ge \eta \implies g(K^* + e) \ge \min\big\{g(K^* - \eta), g(K^* + \eta)\big\} \ge 1.758\,\eta^2 = \varepsilon.$$

The two observations below come from evaluating the cost directly, using the simplification above:

$$g(0.5) = K^* \cdot \frac{1.25}{0.75} - 1 \approx 0.030, \qquad g(0.736) = K^* \cdot \frac{1.5417}{0.9303} - 1 \approx 0.024,$$
$$0.5 - K^* \approx -0.118, \qquad 0.736 - K^* \approx +0.118.$$

- **Observation 1.** A gain about 19% below the optimum has a gap of only about 3%: the gap is quadratic in the perturbation.
- **Observation 2.** Undershooting and overshooting the optimal gain by the same amount, 0.118, give gaps of 0.030 and 0.024: undershooting costs more, because the left wall is closer.

Limits. If the gain of a run settles at a stabilizing value, continuity carries the limit through the gap, so the floor of the learning curve is the gap at the limit gain:

$$k_X(N) \to k_{X,\infty} \in (0, 2) \implies g_X(N) \to g\big(k_{X,\infty}\big).$$

Ordering. The learning curves of the three algorithms may be ordered at every iteration, or only after some iteration:

$$g_A(N) \le g_B(N) \le g_C(N) \quad \forall N, \qquad \exists\, N_0:\ g_A(N) \le g_B(N) \le g_C(N) \quad \forall N \ge N_0.$$

The smallest such iteration is the crossing time. Near the optimal gain the gap is, to leading order, an increasing function of the size of the perturbation, so the ordering of the curves follows the ordering of the perturbations. If the perturbations decay like power laws, the ratio of two of them shows which one wins eventually and when they cross:

$$|e_X(N)| \approx \kappa_X N^{-\beta_X} \implies \frac{|e_X(N)|}{|e_Y(N)|} \approx \frac{\kappa_X}{\kappa_Y}\, N^{-(\beta_X - \beta_Y)} \to 0 \quad \text{if } \beta_X > \beta_Y,$$

$$\frac{\kappa_X}{\kappa_Y}\, N^{-(\beta_X - \beta_Y)} = 1 \iff N = N_{XY} = \left(\frac{\kappa_X}{\kappa_Y}\right)^{1/(\beta_X - \beta_Y)}.$$

So the faster-decaying algorithm is eventually below; if it starts above, it crosses below at the iteration shown.

6 Many seeds

A seed is a random number. All the randomness in one training run, the initial parameters, the exploration noise and any noise in the environment, is generated from it, so once the seed is picked the run is deterministic. The set of seeds Ω is a set of random numbers; in code it is a finite set of integers, which can be pictured as points on a line:

$$\Omega = \{\, 0, 1, 2, \ldots, 2^{32} - 1 \,\} \subset \mathbb{R}, \qquad \omega \in \Omega.$$

Picking a seed is drawing a point from this line, and each point gives one learning curve. Over all of Ω, an algorithm therefore produces a collection of learning curves, one per seed, and the analysis of the algorithm is the analysis of this collection: its average, its spread, its bounds and where it

settles. The question of the notes is whether, as training time increases, the ordering of Section 5 holds for every seed on this line.

The curves are bounded below, because no gain does better than the optimum:

$$J(k) \geq J^* \quad \Longrightarrow \quad g_X(\omega, N) \geq 0 \quad \text{for every seed and iteration.}$$

They are also bounded above whenever the gains of the runs stay in a closed interval inside the stability region, because the gap is continuous there and a continuous function is bounded on a closed interval. So the expected curve below is finite once the runs that leave the stability region are counted separately.

Functions. Seeds are drawn uniformly from the set of integer seeds, and the gain and learning curve become functions of the seed as well as the iteration:

$$\omega \in \Omega, \qquad P(\omega) = 2^{-32},$$

$$\begin{aligned} k_X &: \Omega \times \mathbb{N} \to \mathbb{R}, & k_X(\omega, N) &= \text{gain after } N \text{ iterations with seed } \omega, \\ g_X &: \Omega \times \mathbb{N} \to [0, \infty], & g_X(\omega, N) &= g\big(k_X(\omega, N)\big). \end{aligned}$$

Fixing ω recovers Section 5.

Expected and estimated curves. Over all seeds, and over the M seeds run:

$$\bar{g}_X(N) = \sum_{\omega \in \Omega} P(\omega)\, g\big(k_X(\omega, N)\big), \qquad \hat{g}_X(N) = \frac{1}{M} \sum_{j=1}^{M} g\big(k_X(\omega_j, N)\big).$$

The seeds are independent, so the variance of an average of M curves is the variance of one curve divided by M:

$$\operatorname{Var}\big[\hat{g}_X(N)\big] = \frac{1}{M^2} \sum_{j=1}^{M} \operatorname{Var}\big[g_X(\omega_j, N)\big] = \frac{\sigma_X(N)^2}{M},$$

$$\operatorname{SE}\big[\hat{g}_X(N)\big] = \frac{\sigma_X(N)}{\sqrt{M}}, \qquad \sigma_X(N)^2 = \operatorname{Var}\big[g_X(\cdot, N)\big].$$

Runs whose gain does not stabilize the system are counted as failures and excluded from the averages.

Bias and variance. Averaging the expansion near the optimal gain over seeds gives

$$\bar{g}_X(N) \approx \Big(1 + \frac{2}{\sqrt{5}}\Big)\, \mathbb{E}\big[e_X(\cdot, N)^2\big],$$

$$\mathbb{E}[e^2] = \big(\mathbb{E}[e]\big)^2 + \operatorname{Var}[e], \qquad \mathbb{E}[e] = \mathbb{E}[k] - K^*, \qquad \operatorname{Var}[e] = \operatorname{Var}[k],$$

where the middle identity is the definition of variance rearranged, and the last two hold because the perturbation is the gain shifted by a constant. Together:

$$\bar{g}_X(N) \approx \Big(1 + \frac{2}{\sqrt{5}}\Big)\Big[\big(\mathbb{E}[k_X(\cdot, N)] - K^*\big)^2 + \operatorname{Var}\big[k_X(\cdot, N)\big]\Big].$$

- **Observation 3.** The expected gap splits into squared bias (settling at the wrong gain on average) and variance (inconsistency across seeds).

Ordering across seeds. From strongest to weakest:

$$\max_{\omega} g_A(\omega, N) \le \min_{\omega'} g_B(\omega', N), \qquad \bar{g}_A(N) \le \bar{g}_B(N), \qquad \lim_{N\to\infty} \bar{g}_A(N) \le \lim_{N\to\infty} \bar{g}_B(N),$$

that is, band separation, expected-curve ordering, and floor ordering. Each implies the next. Band separation places every run of A below every run of B, so averaging over both sets of seeds keeps the inequality; taking limits keeps it again:

$$\begin{aligned} & g_A(\omega, N) \le \max_{\omega} g_A(\omega, N) \le \min_{\omega'} g_B(\omega', N) \le g_B(\omega', N) \\ \implies\ & \bar{g}_A(N) \le \bar{g}_B(N) \implies \lim_{N\to\infty} \bar{g}_A(N) \le \lim_{N\to\infty} \bar{g}_B(N). \end{aligned}$$

The converses, the same implications read backwards, fail. Each reversed arrow fails for its own reason, shown by one counterexample.

Floor ordering does not imply expected-curve ordering. A floor depends only on the tail of a curve, so two curves can end in one order after crossing early on:

$$\bar{g}_A(N) = 0.1 + \frac{10}{N}, \qquad \bar{g}_B(N) = 0.2 + \frac{1}{N} : \qquad \lim_{N\to\infty} \bar{g}_A = 0.1 < 0.2 = \lim_{N\to\infty} \bar{g}_B,$$

$$\bar{g}_A(N) > \bar{g}_B(N) \iff 0.1 + \frac{10}{N} > 0.2 + \frac{1}{N} \iff \frac{9}{N} > 0.1 \iff N < 90.$$

Algorithm A starts much worse and overtakes B only after 90 iterations; the floors alone hide this.

Expected-curve ordering does not imply band separation. An average can be ordered while individual runs are not. Suppose two seeds of each algorithm give these gaps at some iteration:

	Seed 1	Seed 2	Average
A	0.01	0.05	0.03
B	0.02	0.06	0.04

The averages are ordered, 0.03 below 0.04, but the worst run of A, 0.05, is above the best run of B, 0.02. The two sets of runs overlap, so not every run of A beats every run of B.

For an experiment, this means each claim must be checked on its own. Ordered floors say nothing about early training, and ordered averages say nothing about individual runs; only band separation supports a statement about every run at every iteration.

7 A possible experiment

- Train A, B and C on the scalar system with reward $r_t = -(x_t^2 + u_t^2)$ and linear policies $u = -kx$; a neural-network variant is evaluated by simulation.
- Keep tuning seeds separate from evaluation seeds.
- Record $k_X(\omega, N)$ and $g_X(\omega, N)$ at checkpoints; count gains outside $(0, 2)$ as failures.
- Report $\hat{g}_X$ with confidence bands, the floors, the bias and variance of the final gains, the crossing times, and which ordering of Section 6 holds.

8 Limitations

- **Discounting.** K^* and J^* depend on γ; compute them with the agents' γ.
- **Noise.** J becomes an expectation; in the scalar case the shape of g is unchanged.
- **Linear policies.** They give the agent the form of the solution; the neural variant tests the realistic case.
- **Locality.** The expansion holds only near K^*.
- **Higher dimensions.** K becomes a matrix and the expansion involves a Hessian.
- **SAC.** Evaluate it by its mean action to avoid the effect of the entropy bonus.

A Proximal Policy Optimization (PPO)

PPO (Schulman et al. 2017) is an on-policy actor-critic algorithm. It alternates between collecting data with the current policy and taking several gradient steps on a clipped objective that keeps the new policy close to the one that collected the data.

Components. A stochastic policy with parameters θ and a value function with parameters ϕ:

$$\pi_\theta(u \mid x), \qquad V_\phi(x).$$

Probability ratio. How much more or less likely the new policy makes an action than the policy that collected it:

$$\rho_t(\theta) = \frac{\pi_\theta(u_t \mid x_t)}{\pi_{\theta_{\text{old}}}(u_t \mid x_t)}.$$

Advantage estimate. The temporal-difference errors of the value function, combined by generalized advantage estimation (Schulman et al. 2016) with discount γ and weight λ:

$$\delta_t = r_t + \gamma V_\phi(x_{t+1}) - V_\phi(x_t), \qquad \hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \, \delta_{t+l}, \qquad \hat{R}_t = \hat{A}_t + V_\phi(x_t).$$

Clipped policy objective. With clipping parameter $\varepsilon_{\text{clip}}$:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t\Big[\min\big(\rho_t(\theta)\,\hat{A}_t,\ \text{clip}\big(\rho_t(\theta),\, 1-\varepsilon_{\text{clip}},\, 1+\varepsilon_{\text{clip}}\big)\,\hat{A}_t\big)\Big].$$

Value loss and entropy bonus.

$$L^V(\phi) = \hat{\mathbb{E}}_t\Big[\big(V_\phi(x_t) - \hat{R}_t\big)^2\Big], \qquad S[\pi_\theta](x_t) = -\int \pi_\theta(u \mid x_t) \log \pi_\theta(u \mid x_t)\, du.$$

Combined objective. Maximized with weights c_1 and c_2:

$$L(\theta, \phi) = \hat{\mathbb{E}}_t\Big[L_t^{\text{CLIP}}(\theta) - c_1 L_t^V(\phi) + c_2\, S[\pi_\theta](x_t)\Big].$$

Algorithm.

1. Initialize the policy parameters θ and the value parameters ϕ.
2. For each iteration:
   (a) Set the data-collecting policy to the current one, $\theta_{\text{old}} \leftarrow \theta$.
   (b) Run the policy for T time steps (in one or more parallel environments) and store the states, controls and rewards.
   (c) Compute the temporal-difference errors, the advantage estimates and the return targets from the stored data.
   (d) For E epochs, split the data into minibatches and, on each minibatch, take a gradient ascent step on the combined objective with respect to θ and ϕ.
3. Return the final policy.

References

- Agarwal, R., Schwarzer, M., Castro, P. S., Courville, A., and Bellemare, M. G. (2021). Deep reinforcement learning at the edge of the statistical precipice. *NeurIPS 2021*.
- Agyei, K., Sarhadi, P., and Polani, D. (2025). Deep reinforcement learning in applied control: Challenges, analysis, and insights. arXiv:2507.08196.
- Fazel, M., Ge, R., Kakade, S. M., and Mesbahi, M. (2018). Global convergence of policy gradient methods for the linear quadratic regulator. *ICML 2018*.
- Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., and Meger, D. (2018). Deep reinforcement learning that matters. *AAAI 2018*.
- Mania, H., Guy, A., and Recht, B. (2018). Simple random search provides a competitive approach to reinforcement learning. arXiv:1803.07055.
- Recht, B. (2019). A tour of reinforcement learning: The view from continuous control. *Annual Review of Control, Robotics, and Autonomous Systems*. arXiv:1806.09460.
- Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. (2016). High-dimensional continuous control using generalized advantage estimation. *ICLR 2016*. arXiv:1506.02438.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. arXiv:1707.06347.
- Tu, S., and Recht, B. (2019). The gap between model-based and model-free methods on the linear quadratic regulator: An asymptotic viewpoint. arXiv:1812.03565.